75.7
Pass

Gaurav

PDF generated at: 24 Feb 2026 17:50:24 UTC

View this report on HackerRank

Score

75.7% • 132.5 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 36 min 32 sec on 24 Feb 2026 22:21:21 IST

Candidate Information

Email	gauravadhav2398@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	24 Feb 2026 22:21:21 IST
Time taken	36 min 32 sec/ 90 min
Work Experience	5 years
Invited by	Vipul
Invited on	24 Feb 2026 19:14:08 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	100%

2	Spring Boot Intermediate	83%
3	Problem Solving Intermediate	100%
4	Java Intermediate	46%

Tags Distribution

Spring Boot	87%	Hard	100%
Spring	75%	JPA	50%
Spring Data	50%	Transactional	50%
Medium	74%	Implementation	100%
Arrays	100%	Algorithms	100%
Problem Solving	100%	Greedy	46%
Sorting	46%	Java	46%

Questions

Spring Boot (Advanced) • 22.5 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Specification Multiple Choice	Spring Boot (Advanced)	5/5	-	1 min 26 sec	✓

2	Cracker Multiple Choice	Spring Boot (Advanced)	5/5	-	3 min 1 sec	
3	Spring Data Transactional Multiple Choice	Spring Boot (Intermediate)	2.5/5	-	6 min 46 sec	
4	Spring Scopes Multiple Choice	Spring Boot (Intermediate)	5/5	-	2 min 20 sec	
5	CSRF Multiple Choice	Spring Boot (Intermediate)	5/5	-	2 min 51 sec	

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Wind Farms Coding	Problem Solving (Intermediate)	75/75	-	12 min 30 sec	

Java (Intermediate) • 35 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: DataVille's Optimal Storage Coding	Java (Intermediate)	35/75	-	5 min 59 sec	

Candidate Feedback

good

1. Specification

Correct

Multiple Choice | Spring Boot | Hard

Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ PreparedStatement
- ● Specification ✓
- ○ Criteria
- ○ Query

No comments.

2. Cracker

Correct

Multiple Choice | Spring Boot | Hard

Question description

Cracker.java

```
@Component
public class Cracker {
    public void startCracking(){
        System.out.println("Cracking...");
    }
    public void stopCracking(){
        System.out.println("Cracking Stopped!");
    }
}
```

CrackerService.java

```
@Service
public class CrackerService {

    @Autowired
    Cracker cracker;
    public void workWithNewCracker() {
        cracker.stopCracking();
        cracker = new Cracker();
    }
    public void startWorking() {
        cracker.startCracking();
    }
}
```

CrackerWorker.java

```
@Service
public class CrackerWorker {

    @Autowired
    CrackerService crackerService;

    public void startWorking() {
        crackerService.startWorking();
        crackerService.workWithNewCracker();
        crackerService = new CrackerService();
        crackerService.startWorking();
```

```
    }
}
```

Consider these classes are managed in Spring Boot. What happens when 'crackerWorker.startWorking()' is called in the Spring context?

Interviewer guidelines

CrackerService in CrackerWorker will work perfectly until it's replaced with new CrackerService. When that happens it just replace existing injected bean with empty pojo. Which doesn't have it's cracker object so when it's startWorking method is called it will try to execute null cracker attributes startWorking method and NullPointerException will arise.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ "Cracking..." "Cracking Stopped!" will be written to console
- ○ "Cracking..." "Cracking Stopped!" "Cracking..." will be written to console
- ● **"Cracking..." "Cracking Stopped!" NullPointerException will be thrown** ✓
- ○ None of above

No comments.

3. Spring Data Transactional

Partially correct

Multiple Choice | Spring | Spring Boot | JPA | Spring Data | Transactional | Medium

Question description

Given the following code, what happens when *StudentRepsitoryTest* runs?

Lesson.java

```
@Entity
public class Lesson {
   @Id
   private Long id;
   private String name;
   private Integer courseLength;
   private Integer courseLevel;
   @JsonBackReference
   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn(name = "student_id")
   private Student student;
   private boolean inProgress;
   //getter & setters
}
```

Student.java

```
@Entity
public class Student {
   @Id
   private Long id;
   private String name;
   private String surname;
   @JsonManagedReference
   @OneToMany(mappedBy = "student", fetch = FetchType.EAGER, cascade = {CascadeType.PERSIST,
CascadeType.REFRESH})
   private List<Lesson> lessons;
   //getter & setters
}
```

StudentRepository.java

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long>{
    //Y
    Optional<Student> findById(Long id);
}
```

LessonRepository.java

```
@Repository
public interface LessonRepository extends JpaRepository<Lesson, Long> {
}
```

StudentService.java

```
@Service
public class StudentService{
    private StudentRepository studentRepository;
    public StudentService(StudentRepository studentRepository, LessonRepository lessonRepository)
{
        this.studentRepository = studentRepository;
        this.lessonRepository = lessonRepository;
    }

    public void save(Student student) {
        studentRepository.save(student);
    }

    //other crud metods

    //X
    public void deleteStudentWithLessons(Long id) throws SQLException {
        Student student =
                studentRepository.findById(id).orElseThrow(() -> new RuntimeException("Student not foun
d"));
        List<Lesson> lessons = student.getLessons();
        lessons.stream().forEach(lesson -> lesson.setStudent(null));
        studentRepository.delete(student);
```

```
        boolean inProgress =lessons.stream()
                .map(Lesson::isInProgress)
                .filter(progress -> progress == true)
                .findFirst()
                .orElseGet(() -> false);
        if (inProgress) {
            throw new SQLException("Cannot delete all Lessons. There are lessons in progress state");
        }
        lessonRepository.deleteAll(lessons);
    }
}
```

StudentRepositoryTest.java

```
@SpringBootTest
class StudentRepositoryTest {
    @Autowired
    StudentRepository studentRepository;
    @Autowired
    StudentService studentService;
    @Autowired
    LessonRepository lessonRepository;
    Student student;
    @BeforeEach
    public void init() {
        student = Student.builder().name("Jorn").surname("Deynhoven").build();
        studentRepository.save(student);
        lessonRepository.saveAll(List.of(
                Lesson.builder().name("Music").courseLength(10).student(student).courseLevel(123).build
(),
                Lesson.builder().name("Art").courseLength(20).courseLevel(123).student(student).build(),
                Lesson.builder().name("Trance").courseLength(30).courseLevel(123).student(student).inPro
gress(true).build()
        ));
    }
    @Test
    void deleteStudentWithLessonsTest() throws SQLException {
        studentService.deleteStudentWithLessons(student.getId());
    }
}
```

Interviewer guidelines

Candidate has to know how to use spring data in any spring application which uses spring data. If an entity wanted to remove from database without removing the child attributes, then first relation between them should be removed. Without doing this, spring will throw DataIntegrityViolationException to keep database consistent. To achive that without changing existing code, we need to annotate deleteStudentWithLessons(Long id) method with @Transactional annotation. With this spring will update the database states of child entities. In this case it will update Lesson attributes to have student as null. Then student will be removed in the next line.

After this there is a inProgress check for existing lessons. it will throw SQLException which is a checked exception. @Transactional annotation in default only rolls back for unchecked exception such as RuntimeException. So all changes will be reflected to database.

Option A and E is right. More info can be found in https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] ** It will try to delete the student then the DataIntegrityViolationException will be thrown. **
- [ ] It will delete the student successfully then an SQLException will be thrown with the message "Cannot delete all Lessons. There are lessons in progress state".
- [x] ** If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException and all database changes will be rolled back. **

- [ ] It will throw a RuntimeException with the message "Student not found".

- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException.

No comments.

4. Spring Scopes

Correct

Multiple Choice | Medium | Spring

Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] Both are used to define the scope of spring beans.
- [x] @Scope("request") creates a bean instance for a single HTTP request.
- [x] @RequestScope can be used instead of @Scope("request").

@Scope("prototype") will return a different instance every time it is requested from the container.

No comments.

5. CSRF

Correct

Multiple Choice | Spring Boot | Medium

Question description

Which Java Spring-based code segment correctly configures CSRF?

Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- private void manage(HttpSecurity http) {   http.csrf().allowed() }

- private void manage(HttpSecurity http) {   http.csrf() }

private void manage(HttpSecurity http) {   http.csrf().enable() }

None of the above. CSRF is configured by default when Spring Security Dependency is added.

No comments.

6. Wind Farms

Correct

Coding | Implementation | Medium | Arrays | Algorithms | Problem Solving

Question description

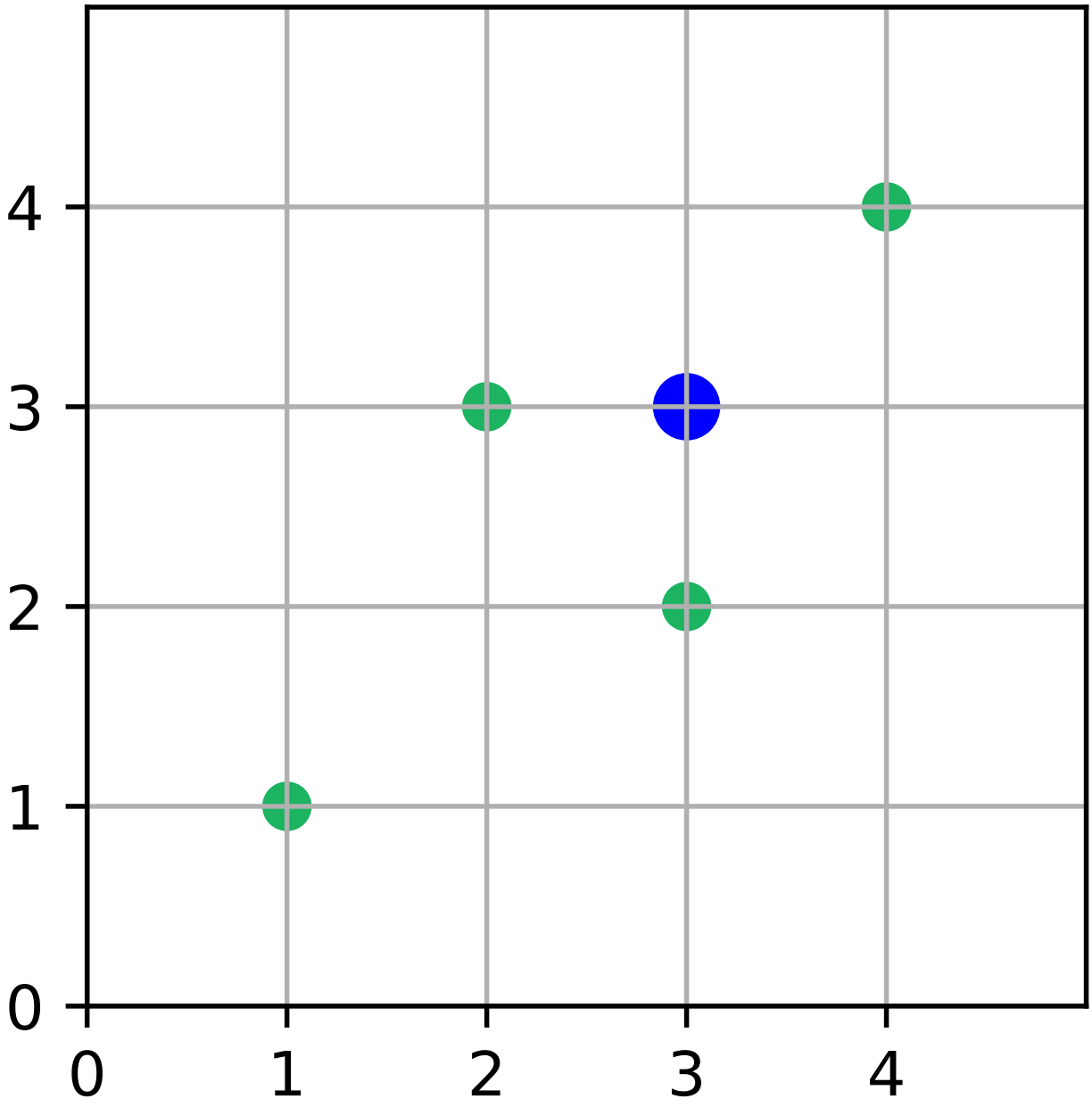
An energy company needs to place a control center to manage a distributed wind farm. Your task is to determine the optimal location for the control center that minimizes the total connection cost.

For each wind farm *i*:

- The farm is located at coordinates *(x[i], y[i])*
- The control center is at coordinates *(cx, cy)*
- The distance is calculated as *|x[i] - cx| + |y[i] - cy|*
- The connection cost is *premium[i] * distance*

Example

x = [1, 3, 2, 4]
y = [1, 2, 3, 4]
premium = [1, 3, 2, 4]



The optimal location is at position [3, 3], resulting in:

Farm #	Location	Distance	Premium	Cost
0	(1, 1)	|1 - 3| + |1 - 3| = 4	1	4 * 1 = 4

1	(3, 2)	|3 - 3| + |2 - 3| = 1	3	1 * 3 = 3
2	(2, 3)	|2 - 3| + |3 - 3| = 1	2	1 * 2 = 2
3	(4, 4)	|4 - 3| + |4 - 3| = 2	4	2 * 4 = 8

The total connection cost is 17, which is the minimum possible.

Function Description

Complete the function *windFarms* in the editor with the following parameter(s):

int premium[n]: the distance multiplier for the cost to connect
int x[n]: *x* coordinates of the wind farms
int y[n]: *y* coordinates of the wind farms

Returns

int: the minimum cost to connect the wind farms

Constraints

- $1 \le n \le 10^3$
- $1 \le x[i], y[i] \le 10^4$
- $1 \le premium[i] \le 50$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer *n*, the size of the array *premium*.
The next *n* lines each contain an element, *premium[i],* where 0 ≤ i < n.
The next line contains an integer *n*, the size of *x[]*.
The next *n* lines each contain an element, *x[i]*.
The next line contains an integer *n*, the size of *y[]*.
The next *n* lines each contain an element, *y[i]*.

Note: *n* is the same value in each instance.

▼ SAMPLE CASE 0

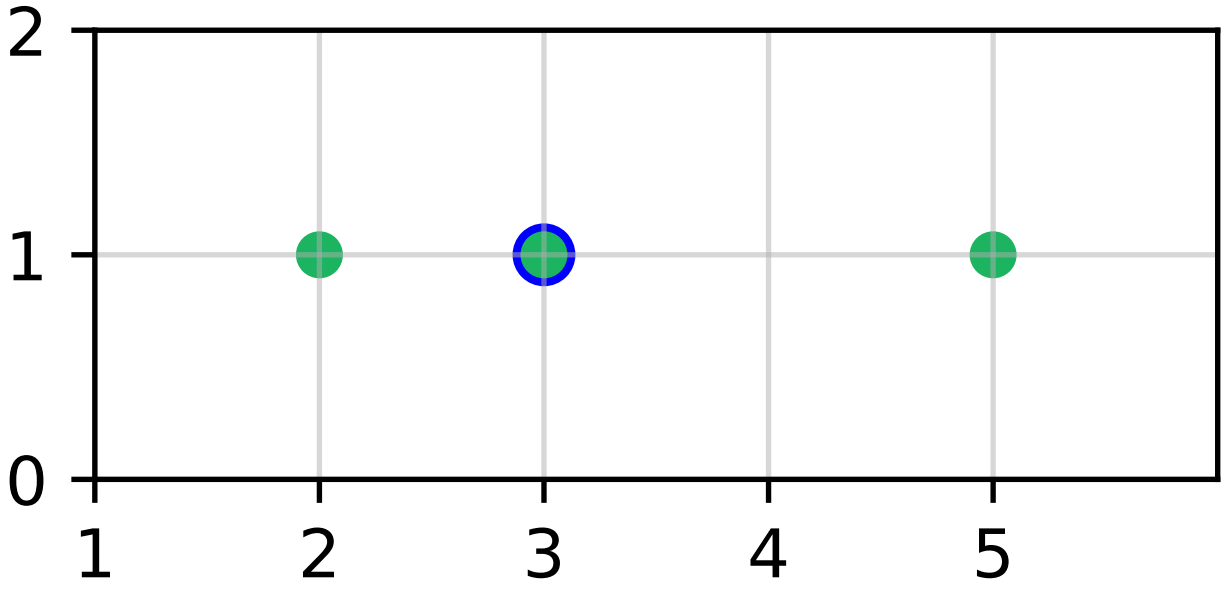
Sample Input 0

```
STDIN    Function
-----    --------
3    →  premium[] size n = 3
1    →  premium = [1, 1, 1]
1
1
3    →  x[] size n = 3
2    →  x = [2, 3, 5]
3
5
3    →  y[] size n = 3
1    →  y = [1, 1, 1]
1
1
```

Sample Output 0

```
3
```

Explanation 0



Locate the controller at farm 1.

Farm #	Location	Distance	Premium	Cost
0	(1, 1)	|1 - 3| + |1 - 1| = 1	1	1 * 1 = 1
1	(3, 1)	|3 - 3| + |1 - 1| = 0	1	0 * 1 = 0

2	(5, 1)	|5 - 3| + |1 - 1| = 2	1	2 * 1 = 2

▼ SAMPLE CASE 1

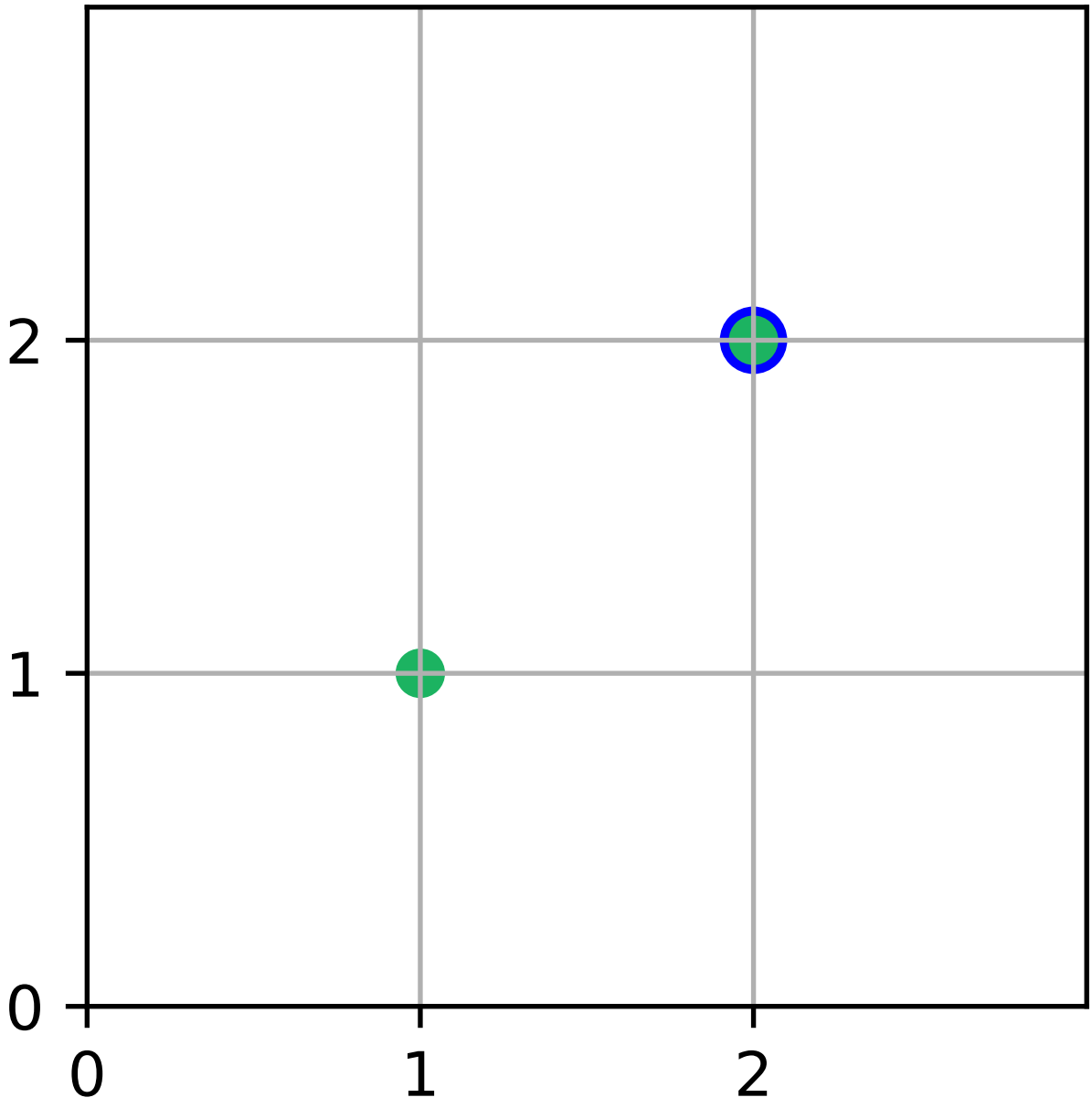
Sample Input 1

```
STDIN    Function
-----    --------
2    →  premium[] size n = 2
1    →  premium = [1, 10]
10
2    →  x[] size n = 2
1    →  x = [1, 2]
2
2    →  y[] size n = 2
1    →  y = [1, 2]
2
```

Sample Output 1

```
2
```

Explanation 1



The sum of the distances to a center at (1, 1), (1, 2), (2, 2), or (2, 1) is 2. Since the premium to connect farm 1 is 10, it is best to locate at position (2, 2) for a cost of 2 * 1 + 0 * 10 = 2.

▼ SAMPLE CASE 2

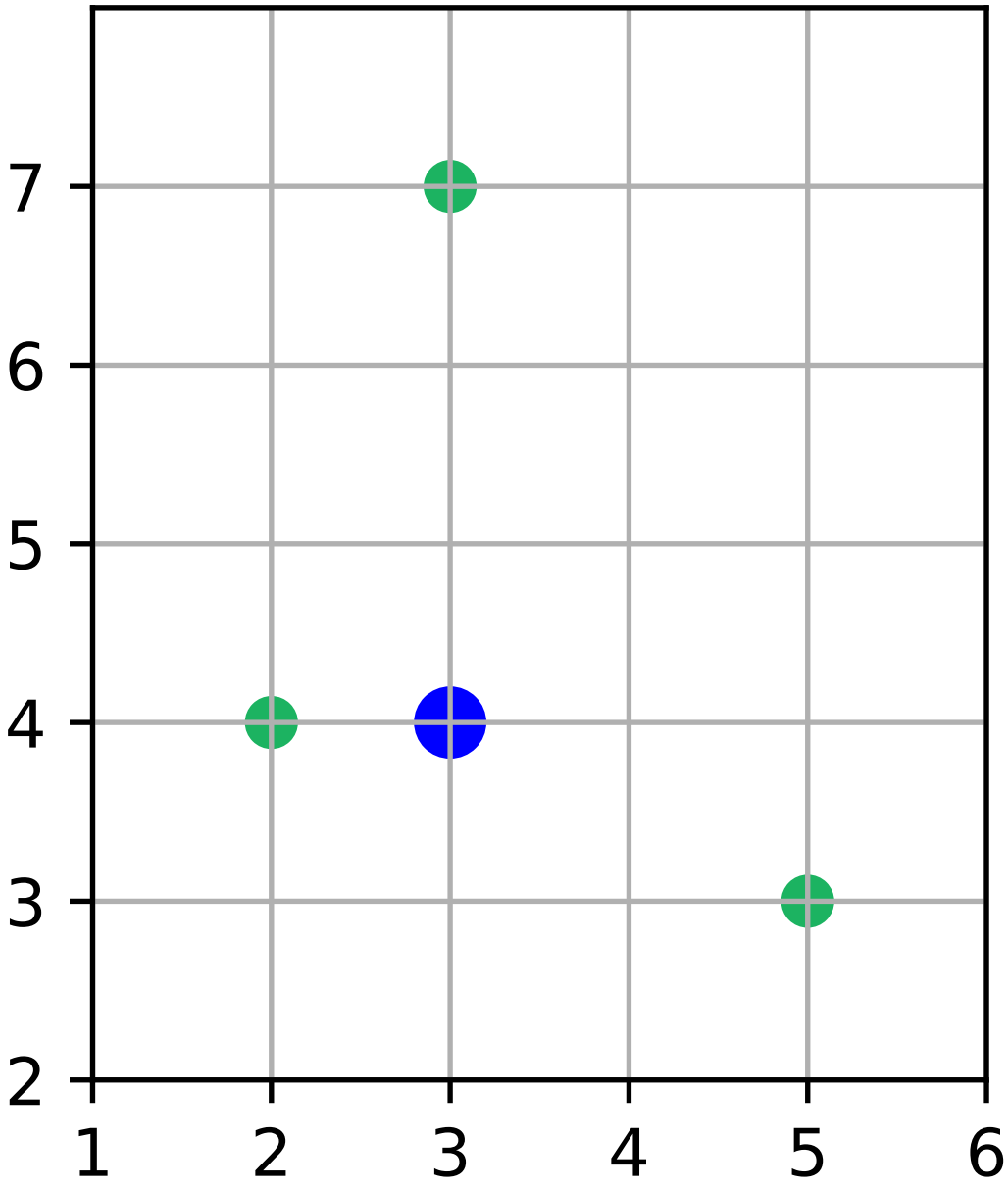
Sample Input 2

```
STDIN    Function
-----    --------
3    →  premium[] size n = 3
1    →  premium = [1, 1, 1]
1
1
3    →  x[] size n = 3
5    →  x = [5, 2, 3]
2
3
3    →  y[] size n = 3
3    →  y = [3, 4, 7]
4
7
```

Sample Output 2

```
7
```

Explanation 2



The optimal location is (3, 4) for a cost of 3 + 1 + 3 = 7.

Interviewer guidelines

▼ SOLUTION

Skills: Loops and counters, I/O

The cost can be calculated for x and y coordinates separately. The function examines each possible x and y coordinate for the control center separately (from 1 to 10000), computes the total cost of connecting all farms for each possible coordinate, and captures the minimum cost found. The ultimate minimum cost is the summation of these minimum costs for x and y coordinates separately.

Optimal Solution

```
def windFarms(premium, x, y):
    minCostX = 10**10
    n = len(x)

    for cx in range(1, 10001):
        cost = 0
        for i in range(n):
            cost += premium[i] * abs(cx - x[i])

        minCostX = min(minCostX, cost)

    minCostY = 10**10
    for cy in range(1, 10001):
        cost = 0
        for i in range(n):
            cost += premium[i] * abs(cy - y[i])

        minCostY = min(minCostY, cost)

    return minCostX + minCostY
```

▼ **COMPLEXITY ANALYSIS**

Time Complexity - O(n)

The universe of x and y values are tested against 10000 values.

Space Complexity - O(1)

Constant space is required regardless of n.

Candidate's Solution

Language used: **Java 21**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;

import org.apache.xml.resolver.apps.resolver;

import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;



class Result {

    /*
     * Complete the 'windFarms' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY premium
     *  2. INTEGER_ARRAY x
     *  3. INTEGER_ARRAY y
     */

    public static int windFarms(List<Integer> premium, List<Integer> x, List<Integer> y) {
```

```
    // Write your code here
    int n = premium.size();
    int cx = weightedMedian(x, premium);
    int cy = weightedMedian(y, premium);
    long totalCost = 0 ;

    for (int i=0;i<n;i++){
        long distance = Math.abs(x.get(i) - cx )+ Math.abs(y.get(i) -cy);
        totalCost+=distance*premium.get(i);

    }
    return (int) totalCost;

    }
    private static int weightedMedian(List<Integer> coords, List<Integer> weights){
        int n = coords.size();
        List<int[]> list = new ArrayList<>();
        for(int i=0; i<n; i++){
            list.add(new int[]{coords.get(i), weights.get(i)});
        }
        list.sort(Comparator.comparingInt(a->a[0]));

        long totalWeight =0;
        for(int w: weights){
            totalWeight+=w;
        }
        long half = (totalWeight+1)/2;
        long cumulative =0;
        for(int[] pair:list){
            cumulative+=pair[1];
            if(cumulative>=half){
                return pair[0];
            }
        }

    return list.get(n-1)[0];
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

```

```
        int premiumCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> premium = IntStream.range(0, premiumCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int xCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> x = IntStream.range(0, xCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int yCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> y = IntStream.range(0, yCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.windFarms(premium, x, y);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

```

```
        bufferedReader.close();
        bufferedWriter.close();
    }
}
public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int premiumCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> premium = IntStream.range(0, premiumCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int xCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> x = IntStream.range(0, xCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int yCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> y = IntStream.range(0, yCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
```

```
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.windFarms(premium, x, y);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0509 sec	37.9 KB
TestCase 1	Easy	Sample	Success	1	0.0569 sec	38 KB
TestCase 2	Easy	Sample	Success	1	0.0503 sec	37.6 KB
TestCase 3	Easy	Sample	Success	2	0.0512 sec	37.7 KB
TestCase 4	Easy	Hidden	Success	2	0.0544 sec	37.8 KB
TestCase 5	Easy	Hidden	Success	2	0.0683 sec	37.4 KB
TestCase 6	Medium	Sample	Success	6	0.0633 sec	38.1 KB
TestCase 7	Medium	Hidden	Success	6	0.0795 sec	38.5 KB

TestCase 8	Medium	Hidden	Success	6	0.0668 sec	38.3 KB
TestCase 9	Hard	Hidden	Success	12	0.1274 sec	42.8 KB
TestCase 10	Hard	Hidden	Success	12	0.1117 sec	43.5 KB
TestCase 11	Hard	Hidden	Success	12	0.085 sec	43 KB
TestCase 12	Hard	Hidden	Success	12	0.0843 sec	43.7 KB

No comments.

7. Java: DataVille's Optimal Storage

Partially correct

Coding Greedy Sorting Medium Java

Question description

In the digital city of DataVille, efficient data management is critical. The city's database is represented as a binary array, *databaseStorage*, containing *n* bits of data. Each bit represents one of two types of data:

- Standard data: Represented by 0.
- Priority data: Represented by 1.

To optimize efficiency during peak operational hours, the number of transitions between 0s and 1s in the array must be minimized. Transitions are points in the array where adjacent elements differ (e.g., a 0 followed by a 1 or a 1 followed by a 0).

Operations:

- You can swap any two bits in the array as many times as you like.

The goal is to rearrange the bits to minimize transitions with the minimum number of swaps.

Example

Consider that *databaseStorage* = [1, 0, 1, 0, 1]. One optimal strategy is to swap *databaseStorage[0]* and *databaseStorage[3]*. After performing this operation, the array is [**0**, 0, 1, **1**, 1], which requires only one transition between *databaseStorage[1]* and *databaseStorage[2]*. This requires only 1 swap, so the return value is 1.

Function Description

Complete the function g*etOptimalStorage* in the editor below.

Function Parameters

int databaseStorage[n]: the current organization of the database's storage, with 0s for standard data and 1s for priority data

Returns

int: the minimum number of operations required to minimize transitions between 0s and 1s

Constraints

- $2 \le n \le 10^5$
- *databaseStorage[i]* $\in$ {0, 1}

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n,* the size of *databaseStorage*.
Each of the next lines contains an integer, *databaseStorage[i].*

▼ SAMPLE CASE 0

Sample Input 0

```
STDIN       FUNCTION
-----       --------
7           databaseStorage[] size n = 7
1           databaseStorage = [1, 0, 1, 1, 0, 0, 1]
0
1
1
0
```

```
0
1
```

Sample Output 0

```
1
```

Explanation

One optimal strategy is to swap *databaseStorage[1]* (which is 0) with *databaseStorage[6]* (which is 1). After performing this operation, the array becomes [1, **1**, 1, 1, 0, 0, **0**]. This solution requires only 1 swap, so the output is 1.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
9         databaseStorage[] size n = 9
1         databaseStorage = [1, 0, 0, 1, 1, 0, 0, 0, 1]
0
0
1
1
0
0
0
1
```

Sample Output 1

```
2
```

Explanation

One optimal strategy of operations is:

1. Swap *databaseStorage*[1] and *databaseStorage*[8]. After performing this operation, the array becomes [1, **1**, 0, 1, 1, 0, 0, 0, **0**].
2. Swap *databaseStorage*[2] and *databaseStorage*[4]. After performing this operation, the array becomes [1, 1, **1**, 1, **0**, 0, 0, 0, 0].

Now there is only one transition. The minimum number of swaps is 2.

Interviewer guidelines

▼ SOLUTION

The solution is to count the number of 0s and 1s in the array. Then, we count the number of 0s and 1s that are out of place. The answer is the minimum of the two counts.

```
class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int zeros = 0, ones = 0;

        for (int brick : databaseStorage) {
            if (brick == 0) {
                zeros++;
            } else {
                ones++;
            }
        }

        int startZero = 0, startOne = 0;

        for (int i = 0; i < ones; ++i) {
            if (databaseStorage.get(i) != 1) {
                startOne++;
            }
        }

        for (int i = 0; i < zeros; ++i) {
            if (databaseStorage.get(i) != 0) {
                startZero++;
            }
        }

        return Math.min(startZero, startOne);
    }
}
```

▼ COMPLEXITY

Time Complexity - O(n)

To traverse over the array, we use a loop that runs n times. The complexity is O(n).

Space Complexity - O(1)

We use only 2 variables, hence the space complexity is O(1).

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int n = databaseStorage.size();
        int totalOnces = 0;
        for(int val:databaseStorage){
            if(val == 1) totalOnces++;
        }
        if(totalOnces==0 || totalOnces == n )return 0;
        int currentOnes = 0;
        for(int i=0;i<totalOnces;i++){
```

```
            if(databaseStorage.get(i)==1){
                currentOnes++;
            }
        }
        int maxOnes = currentOnes;

        for(int i=totalOnces;i<n;i++){
            if(databaseStorage.get(i)==1) currentOnes++;
            if (databaseStorage.get(i-totalOnces)==1) currentOnes--;
            maxOnes = Math.max(maxOnes, currentOnes);
        }
        return totalOnces-maxOnes;

    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int databaseStorageCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> databaseStorage = IntStream.range(0, databaseStorageCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.getOptimalStorage(databaseStorage);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
```

```
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.0563 sec	37.7 KB
TestCase 1	Easy	Sample	Success	1	0.0528 sec	37.6 KB
TestCase 2	Easy	Sample	Success	1	0.051 sec	37.8 KB
TestCase 3	Easy	Hidden	Success	3	0.0531 sec	37.1 KB
TestCase 4	Easy	Hidden	Success	3	0.046 sec	37.2 KB
TestCase 5	Easy	Hidden	Success	3	0.0574 sec	37.6 KB
TestCase 6	Easy	Hidden	Success	4	0.06 sec	37.1 KB
TestCase 7	Easy	Hidden	Success	4	0.0504 sec	37.2 KB
TestCase 8	Easy	Hidden	Success	5	0.0607 sec	37.3 KB
TestCase 9	Easy	Hidden	Success	5	0.0535 sec	37.7 KB

TestCase 10	Easy	Hidden	Success	5	0.0605 sec	37.2 KB
TestCase 11	Medium	Hidden	Wrong Answer	0	0.0855 sec	39.6 KB
TestCase 12	Medium	Hidden	Wrong Answer	0	0.0839 sec	39.5 KB
TestCase 13	Hard	Hidden	Wrong Answer	0	0.0703 sec	40.4 KB
TestCase 14	Hard	Hidden	Wrong Answer	0	0.265 sec	69 KB

No comments.